第一节 数据加载

> 虽然本节内容在实际应用中非常重要，但是由于各种文档加载器的迭代更新，以及各类 AI 应用的不同需求，具体选择需要根据实际情况。本节仅作简单引入，但请务必**重视数据加载**环节，**“垃圾进，垃圾出 (Garbage In, Garbage Out)”** ——高质量输入是高质量输出的前提。

一、文档加载器

在RAG系统中，**数据加载**是整个流水线的第一步，也是不可或缺的一步。文档加载器负责将各种格式的非结构化文档（如PDF、Word、Markdown、HTML等）转换为程序可以处理的结构化数据。数据加载的质量会直接影响后续的索引构建、检索效果和最终的生成质量。

1.1 主要功能

- **文档格式解析**
  将不同格式的文档（如PDF、Word、Markdown等）解析为文本内容。
- **元数据提取**
  在解析文档内容的同时，提取相关的元数据信息，如文档来源、页码等。
- **统一数据格式**
  将解析后的内容转换为统一的数据格式，便于后续处理。

1.2 当前主流RAG文档加载器

工具名称	特点	适用场景	性能表现
PyMuPDF4LLM	PDF→Markdown转换，OCR+表格识别	科研文献、技术手册	开源免费，GPU加速
TextLoader	基础文本文件加载	纯文本处理	轻量高效
DirectoryLoader	批量目录文件处理	混合格式文档库	支持多格式扩展
Unstructured	多格式文档解析	PDF、Word、HTML等	统一接口，智能解析
FireCrawlLoader	网页内容抓取	在线文档、新闻	实时内容获取
LlamaParse	深度PDF结构解析	法律合同、学术论文	解析精度高，商业API
Docling	模块化企业级解析	企业合同、报告	IBM生态兼容
Marker	PDF→Markdown，GPU加速	科研文献、书籍	专注PDF转换

工具名称	特点	适用场景	性能表现
MinerU	多模态集成解析	学术文献、财务报表	集成LayoutLMv3+YOLOv8

二、Unstructured文档处理库

[Unstructured](#) 是一个专业的文档处理库，专门设计用于RAG和AI微调场景的非结构化数据预处理。提供了统一的接口来处理多种文档格式，是目前应用较广泛的文档加载解决方案之一。

2.1 Unstructured的核心优势

格式支持广泛

- 支持多种文档格式：PDF、Word、Excel、HTML、Markdown等
- 统一的API接口，无需为不同格式编写不同代码

智能内容解析

- 自动识别文档结构：标题、段落、表格、列表等
- 保留文档元数据信息

2.2 [支持的文档元素类型](#)

Unstructured能够识别和分类以下文档元素：

元素类型	描述
`Title`	文档标题
`NarrativeText`	由多个完整句子组成的正文文本，不包括标题、页眉、页脚和说明文字
`ListItem`	列表项，属于列表的正文文本元素
`Table`	表格
`Image`	图像元数据
`Formula`	公式
`Address`	物理地址
`EmailAddress`	邮箱地址
`FigureCaption`	图片标题/说明文字
`Header`	文档页眉
`Footer`	文档页脚
`CodeSnippet`	代码片段

元素类型	描述
`PageBreak`	页面分隔符
`PageNumber`	页码
`UncategorizedText`	未分类的自由文本
`CompositeElement`	分块处理时产生的复合元素*

> **注：** `CompositeElement` 是通过分块（chunking）处理产生的特殊元素类型，由一个或多个连续的文本元素组合而成。例如，多个列表项可能会被组合成一个单独的块。

三、从LangChain封装到原始Unstructured

在第一章的示例中，我们使用了LangChain的`UnstructuredMarkdownLoader`，它是LangChain对Unstructured库的封装。接下来展示如何直接使用Unstructured库，这样可以获得更大的灵活性和控制力。

3.1 代码示例

创建一个简单的示例，尝试使用Unstructured库加载并解析一个PDF文件：

> 若代码运行出现报错 `ImportError: libgl.so.1 cannot open shared object file no such file or directory`，执行 `sudo apt-get install python3-opencv` 安装依赖库。

```
from unstructured.partition.auto import partition

# PDF文件路径
pdf_path = "../../data/C2/pdf/rag.pdf"

# 使用Unstructured加载并解析PDF文档
elements = partition(
    filename=pdf_path,
    content_type="application/pdf"
)

# 打印解析结果
print(f"解析完成: {len(elements)} 个元素, {sum(len(str(e)) for e in elements)} 字符")

# 统计元素类型
from collections import Counter
types = Counter(e.category for e in elements)
print(f"元素类型: {dict(types)}")

# 显示所有元素
print("\n所有元素:")
for i, element in enumerate(elements, 1):
    print(f"Element {i} ({element.category}):")
    print(element)
    print("=" * 60)
```

partition函数参数解析：

- `filename`：文档文件路径，支持本地文件路径
- `content_type`：可选参数，指定MIME类型（如"application/pdf"），可绕过自动文件类型检测
- `file`：可选参数，文件对象，与filename二选一使用
- `url`：可选参数，远程文档URL，支持直接处理网络文档
- `include_page_breaks`：布尔值，是否在输出中包含页面分隔符
- `strategy`：处理策略，可选"auto"、"fast"、"hi_res"等
- `encoding`：文本编码格式，默认自动检测

`partition`函数使用自动文件类型检测，内部会根据文件类型路由到对应的专用函数（如PDF文件会调用`partition_pdf`）。如果需要更专业的PDF处理，可以直接使用`from unstructured.partition.pdf import partition_pdf`，它提供更多PDF特有的参数选项，如OCR语言设置、图像提取、表格结构推理等高级功能，同时性能更优。

> **完整代码文件**：`01_unstructured_example.py`

> **Unstructured官方文档**

练习

- 使用`partition_pdf`替换当前`partition`函数并分别尝试用`hi_res`和`ocr_only`进行解析，观察输出结果有何变化。